WEB330: Assignment 9.2 - Governance & AI-Orchestrated Build

STUDENT: Jonathan Cantu
DATE: May 24, 2026

Part I - Governance & Audit Report

Write a 1-2 page report (approximately 400-600 words) that addresses the following:

1) Summary of the Failure

- What went wrong in Week 7?
- What type of issue did AI introduce?
- Why was the failure not immediately obvious?

The AI correctly converted the Promise syntax to async/await and maintained the DOM manipulation logic using the same element IDs. It also properly added try-catch error handling, which is appropriate for async/await patterns. However, the AI fundamentally misunderstood the concurrency model of the original code.

The original Promise.allSettled() initiated all three chef retrievals simultaneously in parallel, allowing them to execute concurrently with a maximum wait time of 1202ms. The AI's refactored version uses await inside a sequential for loop, which forces each chef retrieval to wait for the previous one to complete, resulting in a total wait time of 2704ms.

Evidence of this failure can be observed by timing the page load: the AI version takes more than twice as long to display all chefs. Additionally, the currentChef variable introduces unnecessary shared state that could cause race conditions in more complex scenarios.

The failure was not immediately obvious because no Javascript errors were thrown, all DOM elements populated correctly, console logs look good, visual output matches the original, and no flags were raise.

2) Summary of the Repair

- What change fixed the issue in Week 8?
- Why did that change work?
- What concept was most important in making the fix?

Using `await` with direct data passing:

BEFORE:

```
let currentMovie = null; // shared mutable state

for (let i = 0; i < movies.length; i++) {
  currentMovie = await fetchMovie(i); // sequential blocking
  displayMovie(currentMovie); // uses "shared" variable
}
```

AFTER (Fixed):

```
// Remove shared state variable

const foundMovie = await fetchMovie(title);
displayMovie(foundMovie); // pass data directly
```

The const foundMovie is scoped to the event handler and each form submission creates a new context. The await pauses at the async. Therefore, data can't be overwritten by subsequent calls.

The most important concept here I think is lexical scope and closure capture. In other words, variables are accessible based on where they're defined in the code not where they are executed. When an async completes, it "remembers" the variables from it's creation scope. Understanding that helps you eliminate race conditions, makes async code more predictable and enables proper data flow.

3) Logic Safeguards

Define 3–5 clear rules you would follow when using AI to modify JavaScript code. Each rule must include a brief explanation of why it matters.

Examples (you may define your own):

- Avoid shared variables in async code
- Always verify execution order after AI refactors
- Never assume AI understands timing or dependencies

Well, to start with, AI can't preserve concurrency models - always verify synchronous vs asynchronous execution path. Seems like AI treats syntax as it's primary goal and performance is second. Also, if possible, test multiple simultaneous transactions at once to expose what smaller datasets hide.

Reject AI code that introduces shared mutable state across async boundaries. This creates intermittent race conditions.

AI-modified control flow requires step-wise debugging verification. Early returns, missing awaits, or incorrect error propagation fail silently.

Make sure you look at the performance and timing when validating not just functional testing. Sometimes observing a significant regression in performance during verification can lead to deeper significant issues.

4) Role of AI vs. Role of the Developer

- Where was AI helpful?
- Where was human judgment required?
- What should never be delegated entirely to AI?

Given this two week exercise (weeks 7 and 8), I find it difficult to see anything where AI is helpful in this particular scenario. Perhaps at syntax transformation and pattern recognition? Concerting between promise patterns. No syntax errors (but these are so easy to identify). Saving time on boiler plate code generation. Again, not sure any of this is worth the possibility of introducing race conditions and data flow issues that are difficult to spot.

Human judgement came in where performance degradation was apparent. AI in this instance had no concept of time spent in code execution and treated syntax transformation as a complete success. Verification of shared state safety was another. Deciding on state management patterns. Deciding upon error recovery behavior. Testing, testing, and more testing. Quality assurance is key.

AI should never have the following responsibilities entirely delegated to it:

- Understanding system contracts and implicit arguments (AI doesn't "experience" production failures)
- Defining failure modes and recovery strategies (AI has no concept of business impact)
- Recognizing environmental and state-based situations. (AI doesn't run code in varied environments)
- Maintaining architectural integrity. (AI has no awareness or context of the project history)
- Writing tests that validate assumptions. (AI writes "happy path" tests which pass trivially. It doesn't know what bugs are expensive in the project domain)
- Security and data privacy. (AI doesn't understand domain specific compliance requirements. And security
actually requires adversarial thinking.)
- Production incident response. AI can't read monitoring dashboards or make risk-based decisions.
- Code review and merge decisions. This is dangerous for so many reasons.
- Architecture and design decisions. AI can't balance short-term delivery vs long-term maintainability.
- Performance profiling and optimization. Real performance bottlenecks are often surprising and premature
optimization wastes time on wrong problems.
- Dependency selection and upgrades. These definitely need to be reviewed by a developer and also by security tools and specialists.
- Regression analysis and root cause (without manual intervention and review by an actual developer)

Part II - Guided AI-Orchestrated Build

For this part of the assignment, you will use AI to generate a small JavaScript program. Your responsibility is to guide the AI and validate the result.

1) Program Requirements

The program must:

- Use an array of objects (minimum of 3 items)
- Display information on a web page using the DOM
- Include at least one asynchronous operation (setTimeout, Promise, or async/await)
- Handle success and error cases
- Avoid shared global variables for async data

The program should be similar in size and complexity to assignments you completed earlier in this course.

2) Required AI Prompt (Starting Point)

Use the prompt below as your starting point. You may adjust wording slightly, but you may not expand the scope.

```
Generate a small JavaScript program that:
 - Uses an array of objects
 - Displays data in the browser using the DOM
 - Includes one asynchronous operation
 - Uses async/await correctly
 - Does not rely on shared global variables for async data

Include simple HTML, CSS, and JavaScript files. Keep the code similar in
complexity to introductory JavaScript examples.
```

Do not forget to stage, commit, and push your work to GitHub (i.e., the completed solution).

Final Prompt:

```
Create a Javascript program for Movie Reviews that is 300 lines or less (for a
Intermediate Javascript class). It should include simple HTML, CSS, and
JavaScript files. It needs to fit these criteria:

Use an array of objects (minimum of 3 items)
Display information on a web page using the DOM
Include at least one asynchronous operation (setTimeout, Promise, or async/await)
Handle success and error cases
Avoid shared global variables for async data

As well as meet these guidelines:

• Avoid shared variables in async code
• Verify execution order
• Preserve concurrency - verify synchronous vs asynchronous path.
• Don't introduce shared mutable state across async boundaries. This creates
  intermittent race conditions.
• Maintain proper data flow control. Avoid early returns, missing awaits, and
  incorrect error propagation (no silent failures). Try to provide specific
  errors when they occur for the context of the code.
```

3) Validation & Explanation

After generating the program, submit a validation write-up (12–15 sentences) that explains:

- What the program does
- Where asynchronous behavior occurs
- How execution order is enforced
- Which safeguards from Part I were applied
- What you would verify before using this code in a real project

You may also include brief comments in the code explaining async flow.

It's a simple Movie Review application where the user reviews a pre-defined movie list. Ideally, this should be implemented using Web Storage and have a CSV or text file (with delimited data) so that you can easily update the review list (add, remove, update) based on currently released movies. But, to keep things simple, it's part of the code.

The asynchronous behavior occurs primarily in 2 places:

- saveReviewToServer() {right around line 162}. What makes it async is that it returns a Promise and uses setTimeout to simulate network delay.
- handleReviewSubmit() {Main event handler around line 444}. What makes it async is declared with `async` and uses `await` to pause execution.

Execution order is enforced primarily with the `await` keyword (around line 496).

I provided all of them in the prompt and there are multiple locations throughout the code where comments were provided pointing out where those "guidelines" were enforced.

Again, I don't fully trust automated tests or AI in general. I always say that there's no replacement for manual QA and a user trial period in test environments. More importantly, that you as the developer fully review the code and insure it's integrity and have proper peer review.

Other things you can do or look into regarding verification of web applications of this nature are:

- XSS (Cross Site Scripting - a security flaw)
- SQL injection
- Validation of inputs
- Authentication (make sure different users can't see each other's data)
- Performance profiling
- Error logging
- Accessibility (508 testing)
- Browser compatibility (at least with Chrome, Edge, and Firefox)
- Data persistence across page refreshes
- Rate limiting and abuse prevention (NGINX - throttling)
- Analytics and Monitoring (Grafana, DataDog, CloudWatch, Splunk, etc)

Final Notes and Disclaimers:

It should be noted that there's really less than 300 lines of code if you were to remove all of the comments and spacing throughout. One key observation that I have made about AI generated code is that it goes overboard on commentary. There's some really trivial/obvious things that it comments about that most developers wouldn't bother with. So it looks like quite a bit more code and complication than was requested for the assignment but it's at most marginally more than

requested. Now, as an extra exercise, I did ask Claude to produce tests to cover the "guidelines" (rules from Part I) however I would never fully trust any of this. This is a Movie Review application (nothing mission critical) and your users are probably going to find things that you didn't even consider looking into during verification. However, the higher the bar regarding the importance and impact of a production application, the more stringent the testing and verification should be - including manual QA and even go through pilot testing in an environment with a "user" group (representative of what actual users would be).